

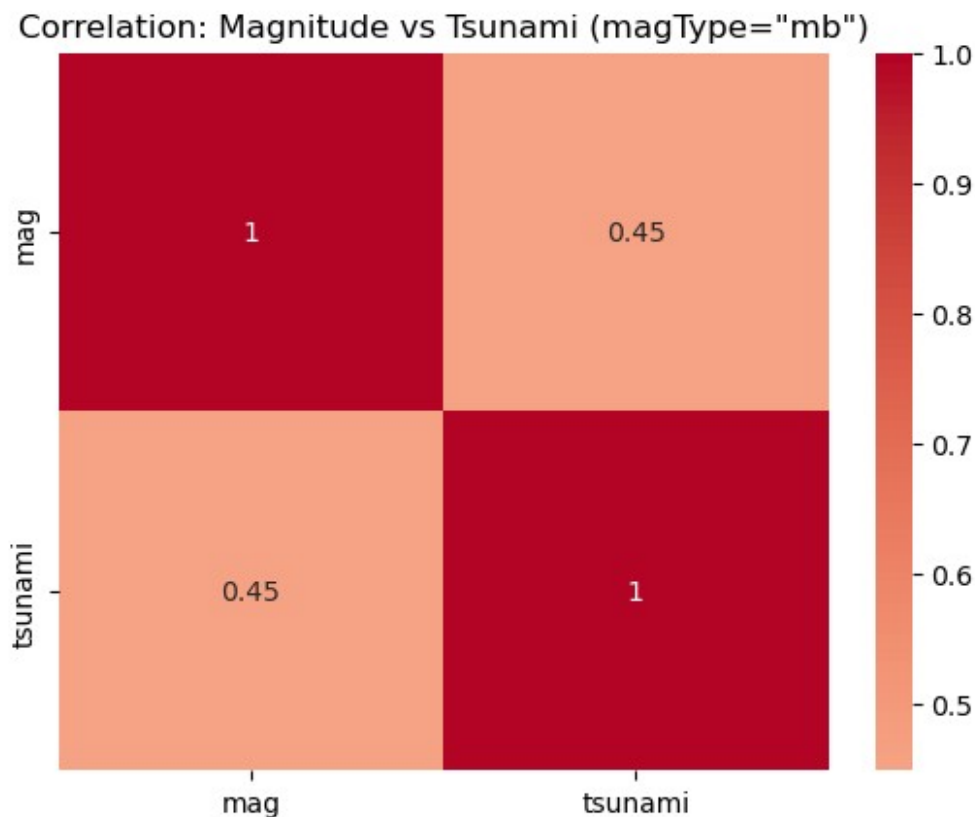
Exercise 1: Earthquake Heatmap

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

try:
    quakes = pd.read_csv('earthquakes-1.csv')

    # Filter for magType 'mb' and select magnitude and tsunami columns
    quakes_mb = quakes[quakes.magType == 'mb'][['mag', 'tsunami']]

    # Create the heatmap of correlation coefficients
    sns.heatmap(quakes_mb.corr(), annot=True, center=0,
cmap='coolwarm')
    plt.title('Correlation: Magnitude vs Tsunami (magType="mb")')
    plt.show()
```



Data Analysis: The heatmap shows the linear relationship between magnitude and tsunamis. A higher positive value would indicate that stronger earthquakes are more likely to cause tsunamis.

Exercise 2 & 3: Facebook Box Plots with Tukey Fences

```
# Load Facebook data
fb = pd.read_csv('fb_stock_prices_2018.csv', index_col='date',
parse_dates=True)

cols = ['volume', 'close']
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for i, col in enumerate(cols):
    # Create the boxplot
    fb[col].boxplot(ax=axes[i])

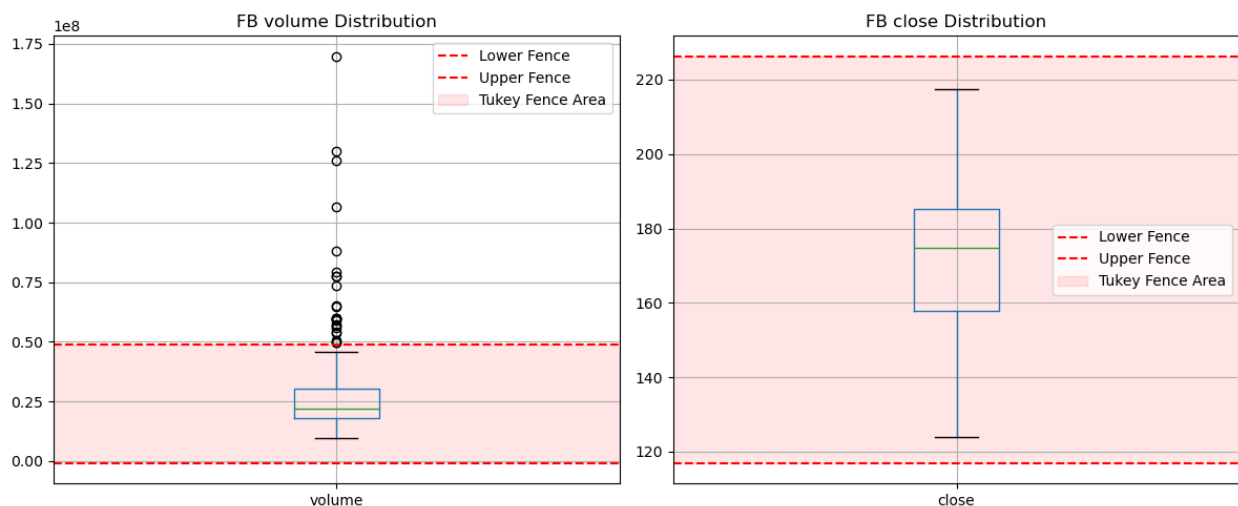
    # Calculate Tukey Fences using quantile()
    q1 = fb[col].quantile(0.25)
    q3 = fb[col].quantile(0.75)
    iqr = q3 - q1
    lower_fence = q1 - 1.5 * iqr
    upper_fence = q3 + 1.5 * iqr

    # Exercise 2: Draw reference lines
    axes[i].axhline(lower_fence, color='red', linestyle='--',
label='Lower Fence')
    axes[i].axhline(upper_fence, color='red', linestyle='--',
label='Upper Fence')

    # Exercise 3: Fill the area between the bounds
    axes[i].axhspan(lower_fence, upper_fence, color='red', alpha=0.1,
label='Tukey Fence Area')

    axes[i].set_title(f'FB {col} Distribution')
    axes[i].legend()

plt.tight_layout()
plt.show()
```



Exercise 4 & 5: Line Plot with Shading and Annotations

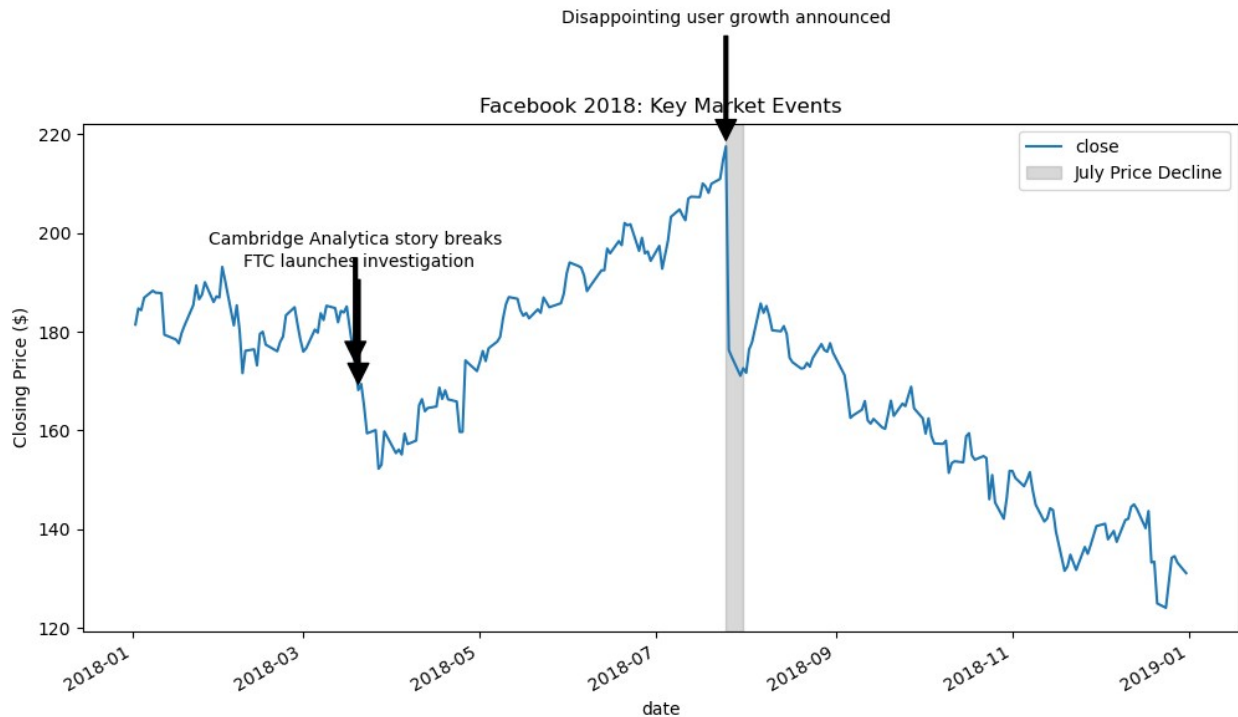
```
# base line plot
ax = fb.close.plot(figsize=(12, 6), title='Facebook 2018: Key Market
Events')

# Exercise 4: Shade the large decline in July using axvspan()
ax.axvspan('2018-07-25', '2018-07-31', color='gray', alpha=0.3,
label='July Price Decline')

# Exercise 5: Annotate three events
events = [
    ('2018-07-25', 'Disappointing user growth announced'),
    ('2018-03-19', 'Cambridge Analytica story breaks'),
    ('2018-03-20', 'FTC launches investigation')
]

for date, text in events:
    y_value = fb.close.loc[date]
    ax.annotate(
        text,
        xy=(date, y_value),
        xytext=(date, y_value + 25), # Position the text above the
point
        arrowprops=dict(facecolor='black', shrink=0.05, width=2),
        ha='center'
    )

plt.ylabel('Closing Price ($)')
plt.legend()
plt.show()
```



Exercise 6: Modifying reg_resid_plots()

```
from matplotlib import cm

def reg_resid_plots(data):
    # Select predictors (all columns except 'close')
    predictors = [col for col in data.columns if col != 'close']
    num_cols = len(predictors)

    # Use a qualitative colormap (Set1)
    cmap = cm.get_cmap('Set1')

    fig, axes = plt.subplots(num_cols, 2, figsize=(10, num_cols * 3))

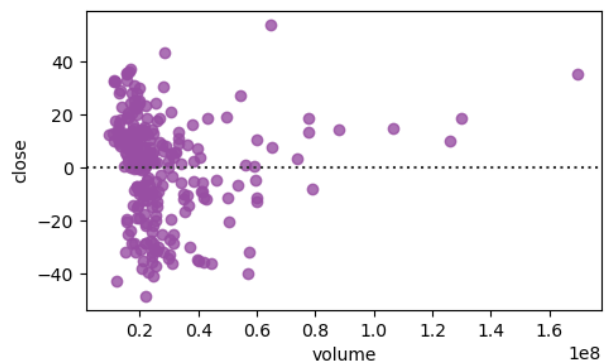
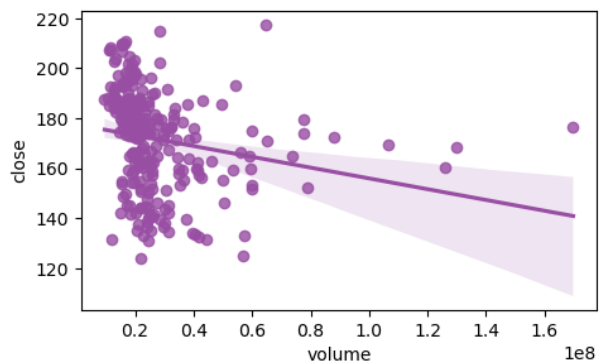
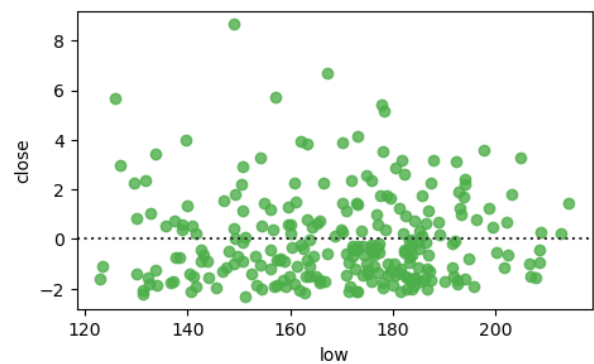
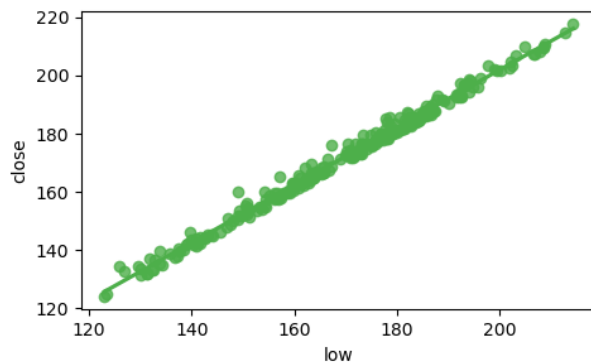
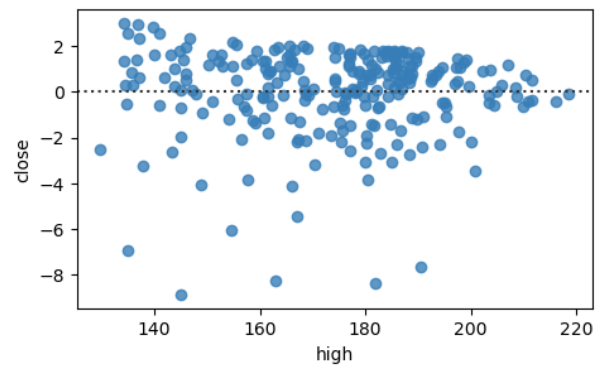
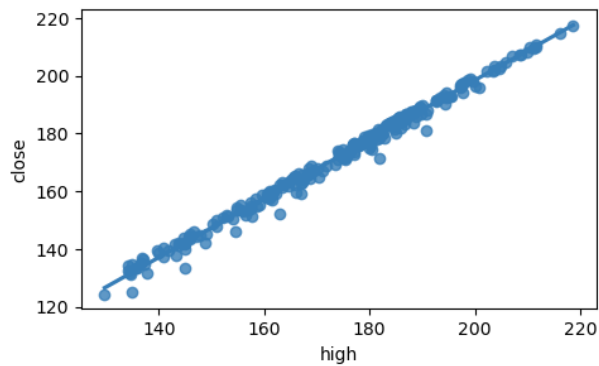
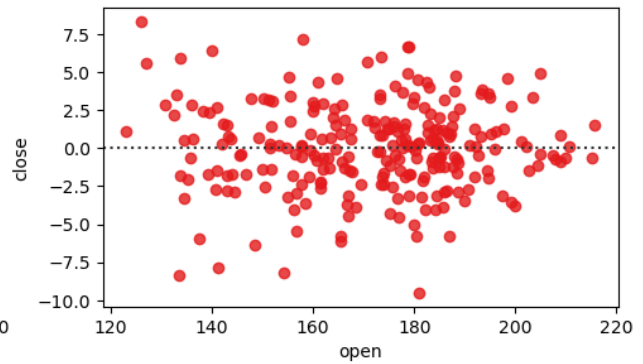
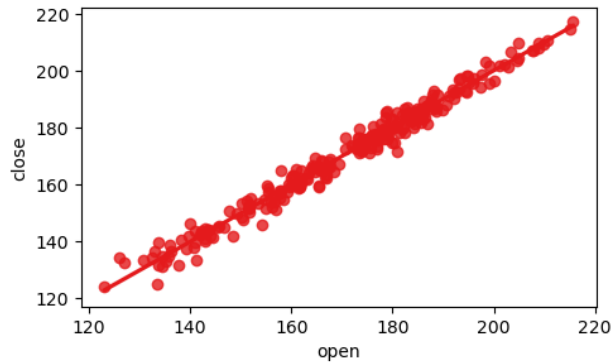
    for i, col in enumerate(predictors):
        color = cmap(i % 9)
        sns.regplot(x=col, y='close', data=data, ax=axes[i][0],
                    color=color)
        sns.residplot(x=col, y='close', data=data, ax=axes[i][1],
                      color=color)

    plt.tight_layout()
    plt.show()

reg_resid_plots(fb[['open', 'high', 'low', 'volume', 'close']])
```

C:\Users\Mariela\AppData\Local\Temp\ipykernel_14992\1635181192.py:9:
MatplotlibDeprecationWarning: The get_cmap function was deprecated in

Matplotlib 3.7 and will be removed in 3.11. Use
`matplotlib.colormaps[name]` or `matplotlib.colormaps.get_cmap()`
or `pyplot.get_cmap()` instead.
`cmap = cm.get_cmap('Set1')`



Summary / Conclusion:

In this activity, I learned how to use visual tools to turn raw data into a clear, professional analysis. By creating **heatmaps**, I was able to quickly identify the correlation between earthquake magnitude and tsunamis, while **scatter matrices** and **lag plots** helped me spot patterns and trends in stock data that aren't obvious in a spreadsheet.

I also practiced using **Tukey Fences** to mathematically define outliers and used `axvspan` to shade the specific week in July when Facebook's stock crashed. Adding **annotations** allowed me to link those price drops to real-world events like the Cambridge Analytica story, providing the "why" behind the numbers. Overall, I've realized that small formatting choices, like choosing distinct **colormaps** or adding reference lines, transform basic charts into a meaningful story that is much easier to interpret.